

Лабораторная работа 6. *Zero-shot* классификация и семантический поиск изображений

В предыдущих лабораторных работах вы работали с одной модальностью — изображениями: классифицировали их, сегментировали, переносили стиль и генерировали из шума. Все эти модели оперировали только визуальной информацией.

В этой работе вы перейдёте к мультимодальным моделям, которые связывают изображения и текст в единое пространство представлений. Это позволяет решать задачи, которые были невозможны в предыдущих работах: классифицировать изображения без обучения и искать изображения по произвольному текстовому описанию.

Вы:

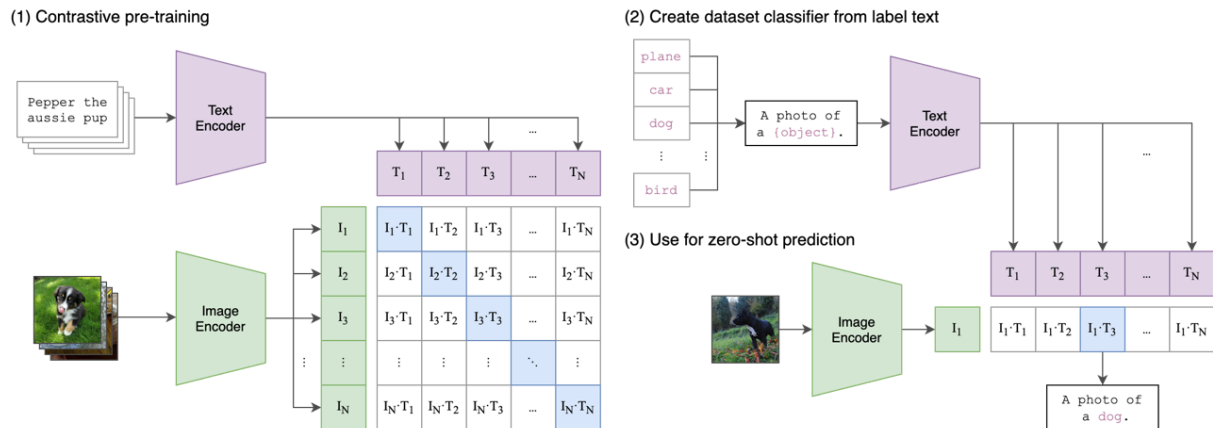
- Освоите модель CLIP — первую и концептуально foundational мультимодальную модель, открывшую эпоху joint vision-language обучения.
- Примените CLIP и убедитесь, что модель может классифицировать изображения без какого-либо обучения — просто по текстовым описаниям классов.
- Построите систему поиска изображений по тексту — введёте произвольный текстовый запрос и получите наиболее подходящие изображения из коллекции.

Теоретическая часть

Как работает CLIP

CLIP (Contrastive Language–Image Pretraining) — модель, разработанная OpenAI в 2021 году, которая обучена на 400 миллионах пар «изображение — текстовое описание» из интернета. CLIP состоит из двух encoder'ов:

- **Image Encoder** — преобразует изображение в вектор (эмбединг) фиксированной размерности.
- **Text Encoder** — преобразует текст в вектор той же размерности.



Оба encoder'а обучены так, чтобы изображение и соответствующее ему текстовое описание имели **близкие** векторы в общем пространстве, а несоответствующие пары — **далёкие**. Близость измеряется через косинусное сходство. Для обучения используется **softmax contrastive loss** — для каждого изображения в батче нужно найти правильное текстовое описание среди всех остальных в батче.

```
"a photo of a cat" → [Text Encoder] → t = [0.2, -0.5, 0.8, ...]
                                                    ↑ cosin
e similarity ≈ высокое
🐱 (фото кошки) → [Image Encoder] → v = [0.3, -0.4, 0.7, ...]

"a photo of a car" → [Text Encoder] → t = [-0.6, 0.1, -0.3, ...]
                                                    ↑ cosin
e similarity ≈ низкое
🐱 (фото кошки) → [Image Encoder] → v = [0.3, -0.4, 0.7, ...]
```

Это позволяет сравнивать изображения и тексты напрямую — без обучения отдельного классификатора.

Эволюция CLIP-подобных моделей

CLIP стал точкой отсчёта, но индустрия не остановилась. Важно понимать текущий ландшафт:

Модель	Организация	Год	Ключевое отличие
CLIP	OpenAI	2021	Softmax contrastive loss; 400M пар
OpenCLIP	LAION / community	2021–н.в.	Open-source, много чекпоинтов вплоть до ViT-G/14; большинство топовых моделей здесь
SigLIP	Google	2023	Sigmoid loss вместо softmax; не требует сравнения всех пар в батче, лучше масштабируется
SigLIP 2	Google	2025	+ multilingual, self-distillation, улучшенный zero-shot

Оригинальный CLIP от OpenAI в современных бенчмарках занимает лишь ~41-е место по средней zero-shot accuracy среди всех доступных CLIP-подобных моделей. На практике для production-использования чаще берут модели из OpenCLIP или SigLIP.

Тем не менее, CLIP — **концептуальная основа** всех этих моделей, и понимание CLIP автоматически даёт понимание всей экосистемы.

Известные ограничения CLIP

Прежде чем работать с моделью, важно понимать её границы:

Ограничение	Описание
Только английский язык	Модель обучалась на англоязычных данных — промпты на других языках дадут непредсказуемые результаты
Fine-grained задачи	CLIP плохо различает похожие подкатегории (породы собак, модели автомобилей, виды птиц)
Счёт и пространственные отношения	На задачах вида «два красных шара слева от синего куба» CLIP лишь незначительно лучше случайного угадывания

Ограничение	Описание
Collapsed embeddings	Визуально разные изображения могут получать очень похожие эмбединги, теряя часть визуальной информации
Смещение датасета	Модель отражает предубеждения и ассоциации, присутствующие в интернет-данных

Эти ограничения вы увидите в ходе выполнения заданий — анализ ошибок модели является частью работы.

Выбор датасета

Требования к датасету

Для этой лабораторной работы датасет должен удовлетворять следующим условиям:

Требование	Почему важно
Задача классификации — датасет должен иметь размеченные категории	Zero-shot классификация требует named классов для составления промптов
Классы имеют смысловые названия на английском	CLIP сравнивает изображения с текстовыми описаниями; классы вроде <code>class_0</code> , <code>class_1</code> бессмысленны
Не менее 3 классов	С двумя классами результаты неинформативны
Не менее 500 изображений в тестовой выборке	Нужно достаточно данных для построения матрицы ошибок и t-SNE
Изображения в формате RGB	Медицинские и спутниковые снимки с нестандартными каналами не подойдут

Вы можете выбрать собственный датасет или выбрать один из предложенных. Ниже приведены датасеты, проверенно работающие с CLIP. Они различаются по **сложности для zero-shot** — это намеренно, чтобы вы могли увидеть, как характер данных влияет на поведение модели:

Датасет	Классов	Размер теста	Сложность для CLIP	Интересная особенность
<code>CIFAR-10</code>	10	10 000	● Низкая	Хорошая baseline,

Датасет	Классов	Размер теста	Сложность для CLIP	Интересная особенность
				удобен для сравнения с CNN
CIFAR-100	100	10 000	● Средняя	Много похожих классов — видны ограничения CLIP
STL-10	10	8 000	● Низкая	Высокое разрешение (96×96), CLIP обучался на похожих данных
Food101	101	25 250	● Средняя	Реальный use-case; классы интуитивно понятны
OxfordIIITPet	37	~3 700	● Высокая	Fine-grained (породы животных)
Flowers102	102	6 149	● Высокая	Fine-grained (виды цветов) — названия классов нетривиальны
EuroSAT	10	5 400	● Средняя	Спутниковые снимки — нестандартный домен для CLIP
GTSRB	43	12 630	● Высокая	Дорожные знаки — интересный тест на fine-grained

Рекомендация: если вы хотите получить интересные результаты для анализа — возьмите датасет со средней или высокой сложностью. На

лёгких датасетах CLIP работает хорошо, но это менее информативно: вы не увидите его ограничений.

Все датасеты из таблицы доступны через `torchvision.datasets`:

```
from torchvision import datasets

# Примеры загрузки – замените на выбранный датасет
dataset = datasets.Food101(root='./data', split='test', download=True)
dataset = datasets.OxfordIIITPet(root='./data', split='test', download=True)
dataset = datasets.Flowers102(root='./data', split='test', download=True)
dataset = datasets.EuroSAT(root='./data', download=True)
dataset = datasets.GTSRB(root='./data', split='test', download=True)
dataset = datasets.STL10(root='./data', split='test', download=True)
```

У разных датасетов разные атрибуты для получения имён классов. Проверьте наличие `dataset.classes` или аналога — для некоторых датасетов список классов придётся задать вручную.

Установка и загрузка модели

В индустрии существует два основных способа работать с CLIP-подобными моделями. Рекомендуется использовать **HuggingFace Transformers** — это современный стандарт, который одинаково работает с CLIP, SigLIP, OpenCLIP и другими моделями.

Способ 1: HuggingFace Transformers (рекомендуется)

```
pip install transformers torch Pillow
```

```
import torch
import numpy as np
from PIL import Image
```

```
import matplotlib.pyplot as plt
from transformers import CLIPModel, CLIPProcessor

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Используется: {device}")

model_id = "openai/clip-vit-base-patch32"
model = CLIPModel.from_pretrained(model_id).to(device)
processor = CLIPProcessor.from_pretrained(model_id)
```

Способ 2: оригинальная библиотека OpenAI

```
pip install git+https://github.com/openai/CLIP.git
```

```
import clip
model, preprocess = clip.load("ViT-B/32", device=device)
```

В этой лабораторной работе мы используем HuggingFace-вариант как основной. Он поддерживает единый API для всех мультимодальных моделей, лучше документирован и является стандартом в индустрии. Оригинальная библиотека OpenAI упомянута для понимания исторического контекста — именно она фигурирует в большинстве старых туториалов и статей.

Проверьте работу модели на простом примере

Убедитесь, что CLIP корректно связывает изображение и текст:

```
image = Image.open("example.jpg").convert("RGB")

descriptions = [
    "a photo of a cat",
    "a photo of a dog",
    "a photo of a car",
]

# Препроцессинг через HuggingFace processor
```

```
inputs = processor(
    text=descriptions,
    images=image,
    return_tensors="pt",
    padding=True
).to(device)

with torch.no_grad():
    outputs = model(**inputs)

    # logits_per_image: [1, num_texts] – сходство изображен
    ия с каждым текстом
    similarity = outputs.logits_per_image.squeeze()

for desc, sim in zip(descriptions, similarity):
    print(f"{sim:.3f} {desc}")
```

Важно: CLIP обучался на англоязычных данных — используйте промпты только на английском языке. Формулировки на других языках могут давать непредсказуемые результаты.

Прокомментируйте результат: какое описание оказалось ближе всего к изображению? Совпадает ли это с вашими ожиданиями?

Задание 1. Обязательное для всех (максимальная оценка — 7)

CLIP: Zero-shot классификация

Часть А. Подготовка датасета

Используйте датасет, выбранный в разделе **Выбор датасета**. Загрузите тестовую выборку.

Важно: препроцессинг изображений выполняется через `processor` от CLIP — не используйте трансформации из предыдущих лабораторных работ.

```
from torch.utils.data import DataLoader
import torchvision.transforms as T
```



```
# Датасет без трансформаций – processor обработает изображение сам
# Замените на ваш выбранный датасет
test_dataset = datasets.Food101(
    root='./data',
    split='test',
    download=True,
    transform=None
)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)
```

Определите текстовые описания классов. Если имена классов есть в атрибуте датасета:

```
# Для большинства torchvision датасетов
class_names = test_dataset.classes
print(f"Классов: {len(class_names)}")
print(f"Примеры: {class_names[:5]}")
```

Убедитесь, что имена классов осмысленны на английском. Для некоторых датасетов (например, GTSRB) классы закодированы числами — вам придётся задать имена вручную или найти маппинг в документации датасета.

Составьте текстовые описания и вычислите текстовые эмбединги:

```
# Стартовый шаблон – вы будете экспериментировать с ним в Части С
text_descriptions = [f"a photo of a {name}" for name in class_names]

text_inputs = processor(
    text=text_descriptions,
    return_tensors="pt",
    padding=True
).to(device)
```

```

with torch.no_grad():
    text_features = model.get_text_features(**text_inputs)
    text_features = text_features / text_features.norm(dim=
-1, keepdim=True)

```

Часть В. Zero-shot классификация

```

all_preds = []
all_labels = []

model.eval()
with torch.no_grad():
    for images, labels in test_loader:

        image_inputs = processor(
            images=list(images),
            return_tensors="pt",
            padding=True
        ).to(device)

        image_features = model.get_image_features(**image_i
nputs)
        image_features = image_features / image_features.no
rm(dim=-1, keepdim=True)

        # [batch_size, num_classes]
        similarity = image_features @ text_features.T
        preds = similarity.argmax(dim=1)

        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.numpy())

all_preds = np.array(all_preds)
all_labels = np.array(all_labels)

```

```
accuracy = np.mean(all_preds == all_labels)
print(f"Zero-shot accuracy: {accuracy * 100:.2f}%")
```

Постройте матрицу ошибок:

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

cm = confusion_matrix(all_labels, all_preds)
disp = ConfusionMatrixDisplay(cm, display_labels=class_names)

# Для датасетов с большим числом классов увеличьте figsize
# и уберите подписи осей – они нечитаемы при 100+ классах
n_classes = len(class_names)
figsize = (max(10, n_classes * 0.3), max(10, n_classes * 0.3))
disp.plot(figsize=figsize, xticks_rotation=45)
plt.title("Confusion Matrix (CLIP zero-shot)")
plt.show()
```

Вычислите accuracy по каждому классу:

```
for i, class_name in enumerate(class_names):
    mask = all_labels == i
    if mask.sum() == 0:
        continue
    class_acc = np.mean(all_preds[mask] == all_labels[mask])
    print(f"{class_name:30s}: {class_acc * 100:.1f}%")
```

Часть С. Эксперименты с промптами

Качество zero-shot классификации CLIP сильно зависит от формулировки текстовых описаний. Исследуйте это влияние.

Попробуйте не менее 4 различных шаблонов промптов:

Шаблон	Пример (для вашего класса)	Accuracy
"{class_name}"	"apple pie"	...%
"a photo of a {class_name}"	"a photo of apple pie"	...%
"a photograph of a {class_name}"	"a photograph of apple pie"	...%
Ваш вариант с контекстом	...	...%
Ваш вариант	...	...%

Для некоторых датасетов полезно добавить контекст домена. Например, для EuroSAT : "a satellite image of {class_name}" вместо "a photo of {class_name}" . Для GTSRB : "a traffic sign showing {class_name}" . Подумайте, какой контекст подходит для вашего датасета.

Попробуйте **ансамблирование промптов**:

```
# Адаптируйте шаблоны под ваш датасет и домен
templates = [
    "a photo of a {}",
    "a photograph of a {}",
    "an image of a {}",
    "a picture of a {}",
    "a {} in a photo",
]

text_features_ensemble = []

for class_name in class_names:
    descriptions = [t.format(class_name) for t in templates]

    text_inputs = processor(
        text=descriptions,
        return_tensors="pt",
        padding=True
    ).to(device)

    with torch.no_grad():
        features = model.get_text_features(**text_inputs)
        features = features / features.norm(dim=-1, keepdim
```

```

=True)
    avg_feature = features.mean(dim=0)
    avg_feature = avg_feature / avg_feature.norm()

    text_features_ensemble.append(avg_feature)

text_features_ensemble = torch.stack(text_features_ensembl
e)

```

Итоговая таблица по промптам:

Шаблон	Accuracy
Худший одиночный шаблон	...%
Лучший одиночный шаблон	...%
Ансамбль (5 шаблонов)	...%

Визуализируйте топ ошибок CLIP. Найдите 6–8 изображений, на которых CLIP ошибается с наибольшей уверенностью. Для каждого покажите:

- Изображение
- Истинный класс
- Предсказанный класс и значение сходства
- Гипотезу: почему CLIP мог ошибиться? Связано ли это с известными ограничениями модели?

Задание 2. Дополнительное — Image Retrieval (максимальная оценка — 10)

Используйте тот же датасет, что и в Задании 1.

В Задании 1 вы использовали CLIP для классификации — сравнения изображений с фиксированным набором текстовых описаний классов. Теперь вы построите более гибкую систему: поиск изображений по произвольному текстовому запросу.

Часть А. Индексирование коллекции изображений

```

all_image_features = []
all_images_raw = []

```

```

all_labels_list = []

model.eval()
with torch.no_grad():
    for images, labels in test_loader:

        image_inputs = processor(
            images=list(images),
            return_tensors="pt",
            padding=True
        ).to(device)

        features = model.get_image_features(**image_inputs)
        features = features / features.norm(dim=-1, keepdim
=True)

        all_image_features.append(features.cpu())
        all_images_raw.append(torch.stack([
            T.ToTensor()(img) for img in images
        ]))
        all_labels_list.append(labels)

all_image_features = torch.cat(all_image_features)
all_images_raw = torch.cat(all_images_raw)
all_labels_list = torch.cat(all_labels_list)

print(f"Проиндексировано изображений: {len(all_image_featur
es)}")
print(f"Размерность эмбединга: {all_image_features.shape
[1]}")

```

```

torch.save({
    'features': all_image_features,
    'labels': all_labels_list,
}, 'image_index.pt')

```

Часть В. Функция поиска

```

def search_by_text(query, image_features, top_k=8):
    text_inputs = processor(
        text=[query],
        return_tensors="pt",
        padding=True
    ).to(device)

    with torch.no_grad():
        text_feat = model.get_text_features(**text_inputs)
        text_feat = text_feat / text_feat.norm(dim=-1, keep
dim=True)

        similarity = (image_features @ text_feat.cpu().T).squeeze()
        top_indices = similarity.argsort(descending=True)[:top_
k]
        top_scores = similarity[top_indices]

    return top_indices, top_scores


def visualize_search_results(query, indices, scores, image
s, labels, class_names):
    n = len(indices)
    fig, axes = plt.subplots(1, n, figsize=(2.5 * n, 3))
    fig.suptitle(f'Запрос: "{query}"', fontsize=14, y=1.02)

    for i, (idx, score) in enumerate(zip(indices, scores)):
        img = images[idx].permute(1, 2, 0).numpy().clip(0,
1)

        axes[i].imshow(img)
        axes[i].set_title(
            f"{class_names[labels[idx]]}\n{score:.3f}",
            fontsize=9
        )
        axes[i].axis('off')

```

```
plt.tight_layout()
plt.show()
```

Часть С. Тестирование системы поиска

Придумайте не менее 8 запросов, **релевантных вашему датасету**, охватывающих разные типы:

Запросы по классу — прямое название объекта из вашего датасета.

Запросы по атрибуту — описание свойств без названия класса:

```
# Примеры для Food101
queries_by_attribute = [
    "something sweet for dessert",
    "a dish with meat",
    "a hot beverage",
]

# Примеры для OxfordIIITPet
queries_by_attribute = [
    "a fluffy animal",
    "a small dog",
    "an animal with pointy ears",
]
```

Абстрактные запросы — нестандартные описания, специфичные для вашей предметной области.

Хорошая система запросов — та, которая осмысленна для вашего датасета. Запрос `"something dangerous"` для Food101 менее интересен, чем `"a dish that looks unhealthy"`. Подумайте, какие запросы раскроют возможности и ограничения CLIP именно на ваших данных.

Часть D. Анализ качества поиска

```
def recall_at_k(class_names, image_features, labels, k=10):
    recalls = {}

    for class_idx, class_name in enumerate(class_names):
```



```

        query = f"a photo of a {class_name}"
        top_indices, _ = search_by_text(query, image_features, top_k=k)

        top_labels = labels[top_indices].numpy()
        hit = int(class_idx in top_labels)
        recalls[class_name] = hit

    mean_recall = np.mean(list(recalls.values()))
    return recalls, mean_recall

recalls_k10, mean_recall = recall_at_k(
    class_names, all_image_features, all_labels_list, k=10
)

print(f"\nRecall@10 по классам:")
for class_name, recall in recalls_k10.items():
    print(f" {class_name:30s}: {'✓' if recall else 'x'}")
print(f"\nMean Recall@10: {mean_recall * 100:.1f}%")

```

Визуализируйте пространство эмбедингов с помощью t-SNE:

```

from sklearn.manifold import TSNE

text_features_np = text_features.cpu().numpy()
image_features_np = all_image_features.numpy()

sample_size = min(2000, len(image_features_np))
sample_idx = np.random.choice(len(image_features_np), sample_size, replace=False)
image_sample = image_features_np[sample_idx]
label_sample = all_labels_list.numpy()[sample_idx]

combined = np.vstack([image_sample, text_features_np])
tsne = TSNE(n_components=2, random_state=42, perplexity=30)
combined_2d = tsne.fit_transform(combined)

image_2d = combined_2d[:sample_size]

```

```

text_2d = combined_2d[sample_size:]

plt.figure(figsize=(12, 10))
scatter = plt.scatter(
    image_2d[:, 0], image_2d[:, 1],
    c=label_sample, cmap='tab10',
    s=5, alpha=0.5, label='Изображения'
)

# Для датасетов с большим числом классов подписи могут пере-
# крываться —
# выведите только часть или используйте интерактивный графич-
# еский (plotly)
for i, class_name in enumerate(class_names[:20]): # ограни-
    чьте при необходимости
    plt.scatter(text_2d[i, 0], text_2d[i, 1],
                s=200, marker='*', color='black', zorder=5)
    plt.annotate(
        class_name,
        (text_2d[i, 0], text_2d[i, 1]),
        fontsize=9, fontweight='bold',
        xytext=(5, 5), textcoords='offset points'
    )

plt.colorbar(scatter)
plt.title("t-SNE: эмбединги изображений и текстовых описан-
ий (CLIP)")
plt.legend()
plt.show()

```

Прокомментируйте: совпадают ли позиции текстовых описаний с кластерами соответствующих изображений? Какие классы оказались близко друг к другу? Ожидаемо ли это семантически?

Для дальнейшей работы (по желанию)

Вы освоили базовый пайплайн работы с CLIP: zero-shot классификацию, поиск по тексту и анализ пространства эмбедингов. Это только точка входа — CLIP-экосистема значительно богаче. В этом разделе собраны

направления, в которых можно двигаться дальше: каждое из них даёт реальный прирост качества или новое понимание того, как работают мультимодальные модели.

Никаких обязательных требований здесь нет. Выбирайте то, что кажется вам интересным.

1. Попробовать более сильную модель

Оригинальный CLIP ViT-B/32 — это намеренно облегчённый вариант, удобный для обучения. В production и исследованиях сегодня используют другие модели.

SigLIP от Google

SigLIP использует **sigmoid loss** вместо softmax contrastive loss из CLIP. Принципиальное отличие: sigmoid loss оценивает каждую пару изображение-текст независимо, не требуя сравнения со всеми остальными парами в батче. Это позволяет обучаться на значительно больших батчах и даёт лучшее качество, особенно на задачах с большим числом классов.

```
from transformers import AutoProcessor, AutoModel

siglip_model = AutoModel.from_pretrained(
    "google/siglip-base-patch16-224"
).to(device)
siglip_processor = AutoProcessor.from_pretrained(
    "google/siglip-base-patch16-224"
)
```

Два важных отличия при использовании SigLIP:

- Шаблон промпта: `"This is a photo of {class}."` вместо `"a photo of a {class}"`
- Обязательный параметр: `padding="max_length"` при обработке текста — без него качество деградирует

```
# Получение эмбеддингов через SigLIP
texts = [f"This is a photo of {name}." for name in class_names]
text_inputs = siglip_processor(
```

```

        text=texts,
        return_tensors="pt",
        padding="max_length"    # обязательно
    ).to(device)

    with torch.no_grad():
        text_features = siglip_model.get_text_features(**text_inputs)
        text_features = text_features / text_features.norm(dim=-1, keepdim=True)

```

Повторите zero-shot классификацию с SigLIP и сравните с CLIP. Обратите внимание: на каких классах разница наибольшая? Совпадает ли это с теми классами, где CLIP ошибался чаще всего?

OpenCLIP — более крупные модели CLIP

Если у вас есть доступ к GPU с достаточным объёмом памяти, попробуйте ViT-L/14 из OpenCLIP — одну из наиболее производительных публично доступных CLIP-моделей:

```

pip install open_clip_torch

```

```

import open_clip

model_ocl, _, preprocess_ocl = open_clip.create_model_and_transforms(
    'ViT-L-14',
    pretrained='laion2b_s32b_b82k'
)
tokenizer_ocl = open_clip.get_tokenizer('ViT-L-14')
model_ocl = model_ocl.to(device)

```

Полный список доступных моделей и весов: `open_clip.list_pretrained()`

2. Linear probe

Zero-shot классификация — это нулевое обучение. Но если у вас есть хотя бы небольшая размеченная выборка, можно сделать следующий

шаг: **заморозить CLIP и обучить только линейный классификатор** поверх эмбедингов.

Идея: CLIP уже научился хорошим признакам. Достаточно обучить финальный слой, чтобы адаптировать эти признаки к вашим конкретным классам.

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import normalize

# Вычислите эмбединги для обучающей выборки
train_features = []
train_labels = []

with torch.no_grad():
    for images, labels in train_loader:
        inputs = processor(
            images=list(images),
            return_tensors="pt",
            padding=True
        ).to(device)
        features = model.get_image_features(**inputs)
        features = features / features.norm(dim=-1, keepdim=True)
        train_features.append(features.cpu().numpy())
        train_labels.extend(labels.numpy())

train_features = np.vstack(train_features)
train_labels = np.array(train_labels)

# Обучаем логистическую регрессию
classifier = LogisticRegression(
    max_iter=1000,
    C=0.316, # стандартный гиперпараметр из оригинальной статьи CLIP
    random_state=42
)
classifier.fit(train_features, train_labels)
```

```
# Оцениваем на тестовой выборке
test_preds = classifier.predict(all_image_features.numpy())
lp_accuracy = np.mean(test_preds == all_labels_list.numpy())
print(f"Linear probe accuracy: {lp_accuracy * 100:.2f}%")
```

Сравните результаты:

Подход	Использует обучающие данные?	Accuracy
Zero-shot CLIP	Нет	...%
Linear probe (CLIP features)	Да, только метки	...%

Вопрос для размышления: как быстро растёт качество linear probe при уменьшении числа обучающих примеров? Попробуйте обучить классификатор на 10%, 1%, 0.1% обучающей выборки — когда zero-shot становится лучше?

3. Улучшение промптов с помощью LLM

В этой работе вы экспериментировали с ручным подбором шаблонов промптов. Но существует более систематичный подход: **сгенерировать описания классов с помощью языковой модели**.

Идея: вместо шаблона `"a photo of a {class}"` попросить LLM написать 10–20 разнообразных описаний для каждого класса — с деталями, контекстом, вариантами внешнего вида. Затем усреднить их эмбединги.

Пример промпта для генерации описаний:

```
Опиши 10 различных способов, которыми может выглядеть фотография {class_name}.
Учитывай разные ракурсы, освещение, фон, контекст.
Каждое описание – одно предложение на английском языке.
```

Этот подход называется **CuPL (Concept-level Prompt Learning)** и даёт заметный прирост качества на fine-grained датасетах — именно там, где стандартные шаблоны работают хуже всего.

4. Эффективный поиск на больших коллекциях

В Задании 2 вы искали изображения перебором — вычисляли сходство со всеми эмбедингами. На коллекции в 10 000 изображений это быстро, но на миллионах — уже нет.

В production для поиска по эмбедингам используют специализированные библиотеки. Одна из них — **FAISS** (Facebook AI Similarity Search):

```
pip install faiss-cpu # или faiss-gpu при наличии CUDA
```

```
import faiss

# Строим индекс
d = all_image_features.shape[1] # размерность эмбединга
index = faiss.IndexFlatIP(d)    # Inner Product = косинусн
                                # ое сходство
                                # (при нормализованных ве
                                # кторах)

# Добавляем все эмбединги
features_np = all_image_features.numpy().astype('float32')
faiss.normalize_L2(features_np) # нормализация на месте
index.add(features_np)

print(f"В индексе: {index.ntotal} векторов")

# Поиск
def search_faiss(query, top_k=8):
    text_inputs = processor(
        text=[query], return_tensors="pt", padding=True
    ).to(device)

    with torch.no_grad():
        text_feat = model.get_text_features(**text_inputs)
        text_feat = text_feat / text_feat.norm(dim=-1, keep
dim=True)

    query_np = text_feat.cpu().numpy().astype('float32')
    faiss.normalize_L2(query_np)

    scores, indices = index.search(query_np, top_k)
    return indices[0], scores[0]
```

Сравните скорость: `search_by_text` vs `search_faiss` на вашем датасете. На каком размере коллекции разница становится ощутимой?

5. Мультимодальные LLM — следующий уровень

CLIP и SigLIP — это **энкодеры**: они умеют сравнивать изображения и тексты, но не умеют описывать изображения словами или отвечать на вопросы о них.

Следующий шаг в развитии мультимодальных моделей — **Vision-Language Models (VLM)**: модели, которые принимают изображение и текстовый запрос и генерируют развёрнутый текстовый ответ.

Модель	Организация	Доступность
PaliGemma 2	Google	HuggingFace, открытые веса
LLaVA	UW / ByteDance / community	HuggingFace, открытые веса
Qwen-VL	Alibaba	HuggingFace, открытые веса
GPT-4o	OpenAI	API
Gemini	Google	API

Ключевое отличие от CLIP: VLM может не просто сказать «это кошка», но и ответить «на изображении рыжая кошка, сидящая на подоконнике, предположительно британской короткошёрстной породы». Это открывает принципиально другие возможности — но и требует значительно больше вычислительных ресурсов.

Пример запуска небольшой VLM через HuggingFace:

```
from transformers import PaliGemmaProcessor, PaliGemmaForConditionalGeneration

model_id = "google/paligemma2-3b-mix-224"

vlm = PaliGemmaForConditionalGeneration.from_pretrained(
    model_id,
    torch_dtype=torch.bfloat16,
    device_map="auto"
).eval()
vlm_processor = PaliGemmaProcessor.from_pretrained(model_id)
```



```

image = Image.open("example.jpg").convert("RGB")

prompt = "Describe this image in detail."

model_inputs = vlm_processor(
    text=prompt,
    images=image,
    return_tensors="pt"
).to(torch.bfloat16).to(vlm.device)

input_len = model_inputs["input_ids"].shape[-1]

with torch.inference_mode():
    generation = vlm.generate(**model_inputs, max_new_tokens=100, do_sample=False)

generation = generation[0][input_len:]
decoded = vlm_processor.decode(generation, skip_special_tokens=True)
print(decoded)

```